TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN — ĐIỆN TỬ

BÁO CÁO TIẾN ĐỘ ĐỒ ÁN CHUYÊN NGÀNH

Đề tài 14 — Thiết kế mạng WiFi Mesh kết nối Drone và Robot

(SmartNetworkX & SmartIoTX)

Sinh viên thực hiện: Nguyễn Dương Hiếu
Ngành: Kỹ thuật Điện tử — Viễn thông
Nội dung: Sơ đồ khối mô hình, lưu đồ giải thuật và báo cáo tiến độ

Ngày lập báo cáo: 18/09/2026

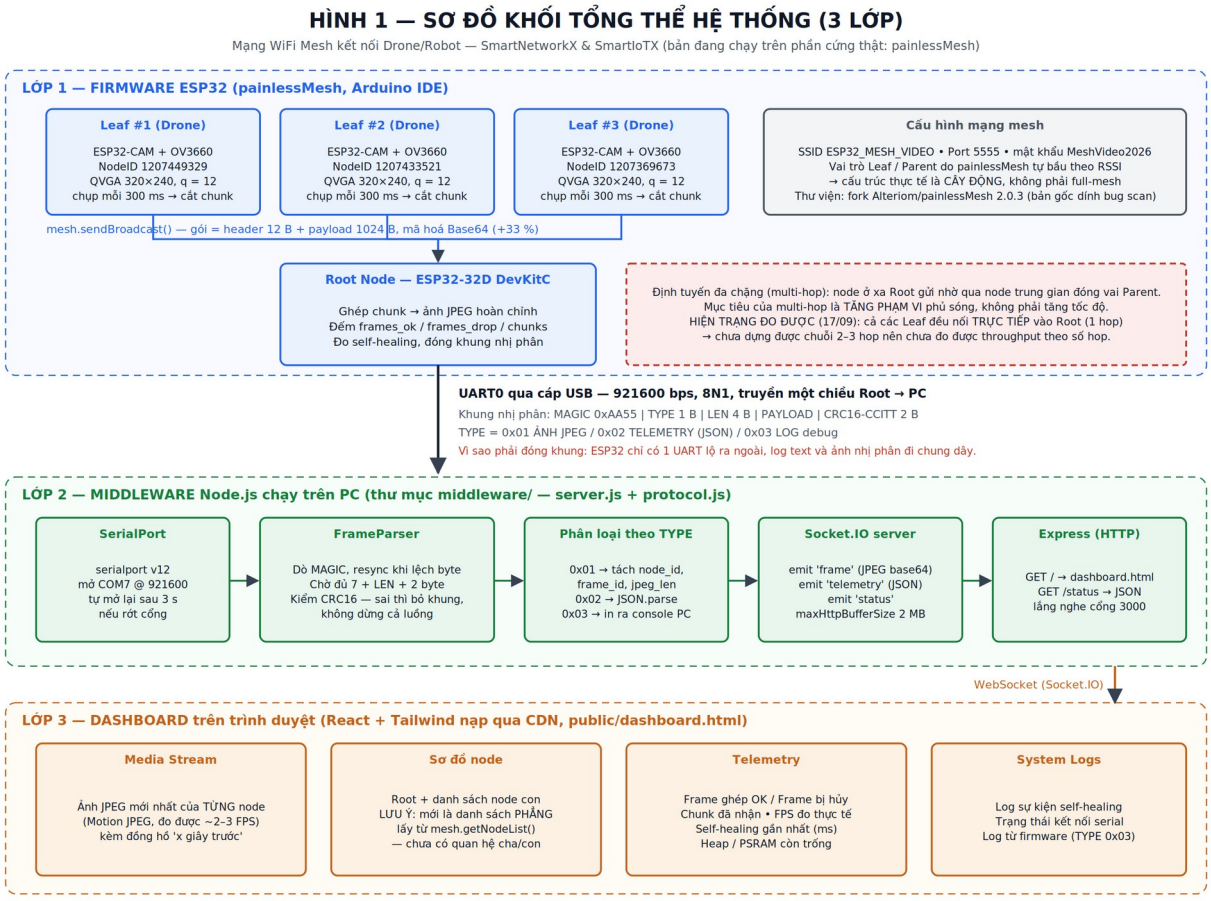
1. Phạm vi và cách đọc bản báo cáo này

Bản báo cáo tổng hợp toàn bộ công việc đã thực hiện cho Đề tài 14 tính đến ngày 18/09/2026, dựa trên hai nguồn dữ liệu: mã nguồn thực tế đang nằm trong thư mục D:\DOAN_CHUYENNGANH_MESH trên máy thực hiện, và năm nhật ký làm việc đã ghi trong thư mục nhatkylamviec (từ 12/09 đến 17/09). Mọi con số nêu trong báo cáo đều là số đo thật hoặc trích dẫn trực tiếp từ mã nguồn; những phần chưa có số liệu được ghi rõ là chưa đo được thay vì ước lượng.

Cần nhấn mạnh ngay từ đầu bản chất của đề tài: đây không phải một đồ án lập trình ESP32 chụp ảnh và gửi qua WiFi, mà là một đồ án chứng minh và đo lường hành vi của một mạng vô tuyến đa chặng khi mạng tự tổ chức lại chính nó. Hội đồng Điện tử Viễn thông không chấm phần cơ khí mà chấm ba chỉ số của mạng: throughput thực tế suy giảm ra sao theo số chặng, độ trễ từ lúc gửi đến lúc nhận, và thời gian mạng tự định tuyến lại khi một node trung gian bị mất. Vì vậy phần 8 và phần 9 của báo cáo — nói về những gì còn thiếu và cách khắc phục — quan trọng không kém phần trình bày những gì đã làm được.

Báo cáo có bảy hình. Hình 1 đến Hình 3 mô tả kiến trúc: sơ đồ khối tổng thể ba lớp, sơ đồ khối phần cứng một node, và cấu trúc gói tin. Hình 4 đến Hình 6 là lưu đồ giải thuật của ba khối phần mềm chính. Hình 7 tổng hợp tiến độ. Tất cả các hình đều được vẽ lại từ chính mã nguồn hiện hành, kèm sẵn phần ghi chú giải thích để dùng trực tiếp khi trình bày trước hội đồng.

2. Sơ đồ khối tổng thể mô hình hệ thống



Hình 1 — Sơ đồ khối tổng thể hệ thống, ba lớp Firmware / Middleware / Dashboard

Hệ thống được chia thành ba lớp có ranh giới rõ ràng. Lớp thứ nhất là firmware chạy trên các board ESP32. Ba board ESP32-CAM gắn camera OV3660 đóng vai Drone, gọi là node Leaf; chúng chụp

ảnh JPEG ở độ phân giải QVGA 320×240 với mức nén quality 12, cứ 300 ms một lần, rồi cắt ảnh thành các mảnh 1024 byte và phát vào mạng mesh. Một board ESP32-32D DevKitC đóng vai Root, không có camera, nhiệm vụ duy nhất là thu gom các mảnh, ghép lại thành ảnh hoàn chỉnh và đẩy lên máy tính.

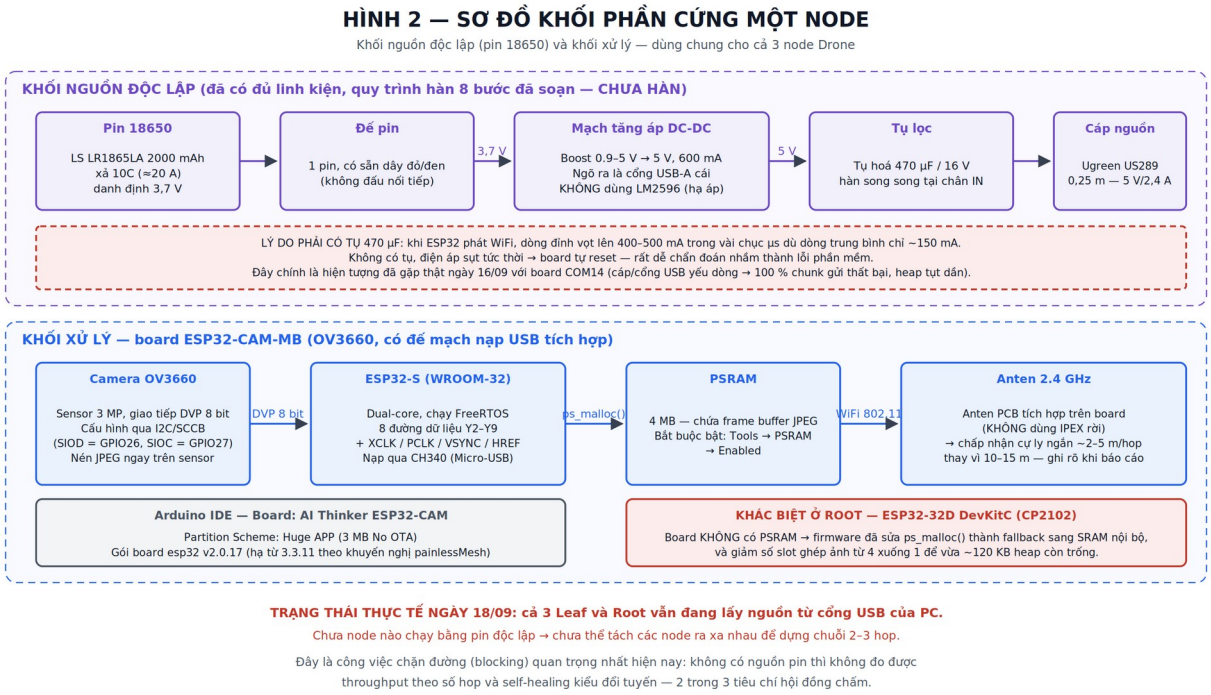
Điểm cần nói rõ khi bảo vệ là bản chất tô-pô của mạng. Thư viện painlessMesh không tạo ra một mạng lưới đầy đủ (full mesh) mà tạo ra một cây động: mỗi node tự chọn một node cha dựa trên cường độ tín hiệu RSSI, và khi tín hiệu với node cha hiện tại yếu đi thì node quét lại rồi chuyển sang node cha khác có đường về Root. Vai trò Leaf hay Parent vì thế không phải do phần cứng quy định mà do vị trí vật lý lúc chạy quyết định — đây chính là lý do trong danh sách linh kiện không có board nào được mua riêng để làm Parent.

Lớp thứ hai là middleware viết bằng Node.js chạy trên máy tính, nằm trong thư mục middleware. Root gửi dữ liệu xuống máy tính qua đúng một đường UART ở tốc độ 921600 bps. Tốc độ này được chọn thay cho mức mặc định 115200 với lý do rất cụ thể: nếu để 115200, riêng đường UART đã chỉ chuyển được khoảng 11 KB mỗi giây, tức là không đủ cho một ảnh 10–15 KB ở tốc độ 2–3 khung hình mỗi giây, và đường UART sẽ trở thành nút thắt cổ chai làm sai lệch chính phép đo throughput của mạng mesh. Middleware đọc luồng byte thô, tách khung, rồi phát lại cho trình duyệt qua WebSocket.

Lớp thứ ba là dashboard chạy trên trình duyệt, dựng bằng React và Tailwind nạp qua CDN, được chính middleware phục vụ tại địa chỉ localhost cổng 3000. Dashboard hiển thị bốn khối: luồng ảnh của từng node, sơ đồ node, các thông số telemetry và nhật ký sự kiện. Cần ghi nhận một hạn chế trung thực ở đây: khối sơ đồ node hiện chỉ vẽ được danh sách phẳng các node đang kết nối lấy từ hàm mesh.getNodeList(), chưa thể hiện được quan hệ cha con, vì painlessMesh không bộc lộ thông tin này qua giao diện lập trình công khai một cách ổn định giữa các phiên bản. Dashboard hiện vẽ đúng những gì có thật thay vì vẽ một cây suy diễn.

Về lựa chọn WebSocket thay vì MQTT: kiến trúc hiện tại chỉ có một Root, một middleware và một dashboard, nên không cần một broker trung gian. WebSocket cho độ trễ thấp hơn và đơn giản hơn nhiều khi triển khai trong thời gian ngắn. MQTT chỉ trở nên hợp lý nếu sau này mở rộng ra nhiều Root hoặc nhiều dashboard cùng lúc — đây là một hướng phát triển đáng nêu trong phần kết luận của báo cáo cuối kỳ.

3. Sơ đồ khối phần cứng một node



Hình 2 — Sơ đồ khối phần cứng một node: khối nguồn độc lập và khối xử lý

Mỗi node Drone gồm hai khối. Khối nguồn đi từ một viên pin 18650 LR1865LA 2000 mAh loại xả cao, qua đế pin, vào mạch tăng áp DC-DC 0,9–5 V lên 5 V. Ở đây có một quyết định kỹ thuật cần giải thích được khi hội đồng hỏi: bắt buộc phải dùng mạch tăng áp chứ không phải mạch hạ áp kiểu LM2596, vì một viên 18650 chỉ cho khoảng 3,7 V, thấp hơn mức 5 V mà board cần, nên mạch hạ áp không thể hoạt động.

Tụ hoá 470 µF hàn song song ngay tại chân đầu vào của mạch boost cũng không phải chi tiết trang trí. Khi ESP32 phát WiFi, dòng tiêu thụ đỉnh vọt lên 400–500 mA trong vài chục micro giây dù dòng trung bình chỉ khoảng 150 mA; nếu nguồn không đáp ứng kịp, điện áp sụt tức thời và board tự khởi động lại. Hiện tượng này rất dễ bị chẩn đoán nhầm thành lỗi phần mềm, và trên thực tế nhóm đã gặp đúng tình huống đó ngày 16/09: board cắm ở cổng COM14 báo thất bại 100 % khi gửi mảnh dữ liệu, bộ nhớ heap tụt dần từ 154 KB xuống 72 KB sau khoảng 190 khung hình; chỉ cần đổi sang cổng và cáp USB khác là hết lỗi, heap ổn định trở lại. Đây là một bài học đáng đưa vào báo cáo vì nó minh hoạ đúng ranh giới giữa lỗi nguồn và lỗi phần mềm.

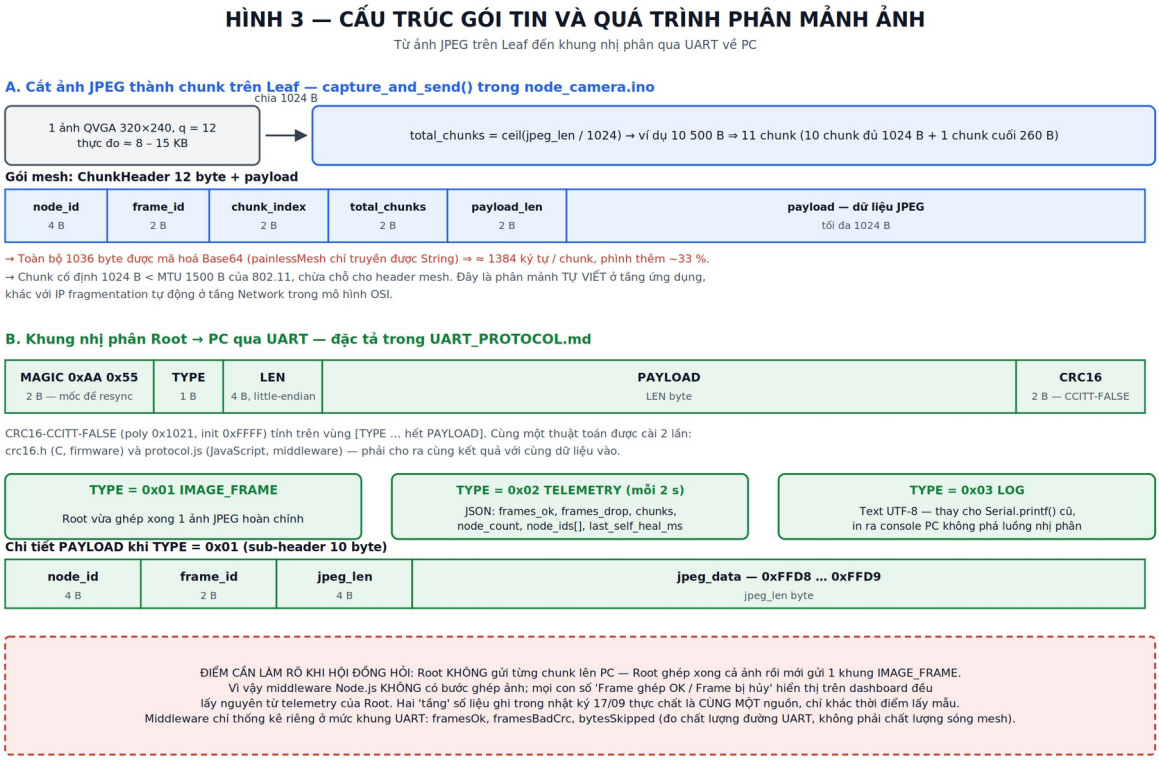
Khối xử lý là board ESP32-CAM-MB: cảm biến OV3660 nối với ESP32 qua giao tiếp song song DVP 8 bit và được cấu hình qua bus I2C/SCCB; ảnh được nén JPEG ngay trên cảm biến nên ESP32 không phải tự nén. Bộ nhớ PSRAM 4 MB dùng để chứa frame buffer ảnh, và bắt buộc phải bật tuỳ chọn PSRAM trong Arduino IDE. Anten là loại PCB tích hợp sẵn trên board, không dùng anten IPEX rời — đây là lựa chọn có chủ ý vì mô hình chỉ là chứng minh nguyên lý, nhưng hệ quả là cự ly mỗi chặng thu hẹp còn khoảng 2–5 mét thay vì 10–15 mét, và giới hạn này phải được ghi rõ trong phần Giới hạn của mô hình thực nghiệm.

Board Root có một khác biệt quan trọng đã phải sửa mã nguồn để xử lý: board ESP32-32D DevKitC thực tế mua về không có PSRAM vật lý, trong khi mã nguồn ban đầu giả định có PSRAM và còn có một khối kiểm tra chặn cứng khiến board treo vĩnh viễn ngay trong hàm setup(). Ngày 12/09 đã sửa bằng cách đổi cấp phát bộ nhớ sang cơ chế ưu tiên PSRAM nhưng tự động lùi về SRAM nội bộ, bỏ

khối chặn cứng, và giảm dung lượng bộ đệm cho vừa heap thực tế. Chính lần giảm dung lượng này lại sinh ra một hệ quả phụ nghiêm trọng sẽ phân tích ở mục 6.2.

Trạng thái phần cứng tại thời điểm báo cáo: chưa hàn bộ nguồn pin nào. Cả ba Leaf và Root vẫn đang lấy điện từ cổng USB của máy tính, nghĩa là các node bị buộc phải nằm trong phạm vi chiều dài sợi cáp — cách nhau vài chục centimét trên cùng một mặt bàn. Đây là nguyên nhân gốc khiến chưa thể dựng được chuỗi hai hoặc ba chặng để đo throughput theo số chặng.

4. Cấu trúc gói tin và cơ chế phân mảnh ảnh



Hình 3 — Cấu trúc gói tin ở tầng mesh và khung nhị phân ở tầng UART

Một ảnh JPEG QVGA ở mức nén quality 12 đo thực tế nằm trong khoảng 8 đến 15 KB, trong khi đơn vị truyền lớn nhất của chuẩn 802.11 chỉ khoảng 1500 byte. Vì vậy ảnh phải được cắt nhỏ trước khi gửi. Đề tài chọn kích thước mảnh cố định 1024 byte, nhỏ hơn hẳn 1500 byte để chừa chỗ cho phần tiêu đề của mesh và để giảm rủi ro hỏng gói khi sóng yếu. Cần phân biệt rõ với lý thuyết đã học: đây là phân mảnh tự viết ở tầng ứng dụng, khác với phân mảnh IP tự động ở tầng Network trong mô hình OSI.

Mỗi mảnh mang một tiêu đề 12 byte gồm năm trường: node_id bốn byte để Root biết mảnh này của camera nào, frame_id hai byte là số thứ tự ảnh, chunk_index hai byte cho biết mảnh nằm ở vị trí nào, total_chunks hai byte cho biết ảnh có tất cả bao nhiêu mảnh, và payload_len hai byte vì mảnh cuối cùng thường ngắn hơn 1024 byte. Nhờ có chunk_index, Root tính được vị trí ghi trực tiếp bằng công thức offset bằng chunk_index nhân 1024, nên các mảnh đến không đúng thứ tự vẫn được ghép chính xác.

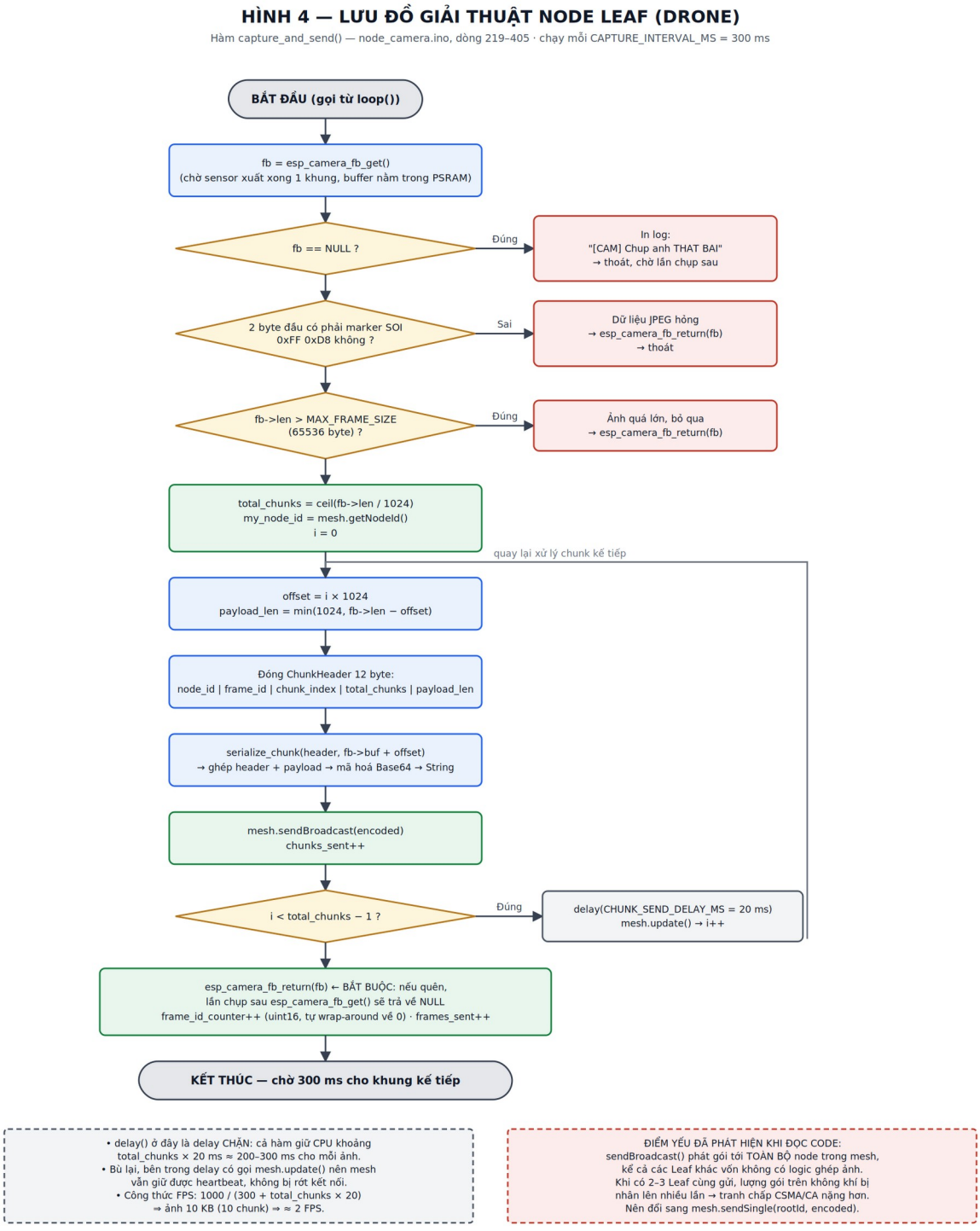
Có một khoản chi phí phải thừa nhận thẳng thắn: painlessMesh chỉ truyền được chuỗi ký tự, nên toàn bộ 1036 byte nhị phân phải mã hoá Base64 thành khoảng 1384 ký tự trước khi gửi, tức là phình thêm khoảng 33 % lượng dữ liệu phải truyền trên không khí. Đây là một trong những lý do chính khiến bản chuyển sang ESP-WIFI-MESH được chuẩn bị song song, vì giao diện esp_mesh_send cho phép gửi dữ liệu nhị phân trực tiếp và bỏ được hoàn toàn lớp Base64 này.

Ở tầng UART giữa Root và máy tính, mọi thứ đi ra khỏi Root — kể cả dòng log gỡ lỗi — đều được đóng vào một khung nhị phân có cấu trúc MAGIC hai byte, TYPE một byte, LEN bốn byte, PAYLOAD và cuối cùng là CRC16 hai byte. Lý do bắt buộc phải làm như vậy: ESP32-CAM chỉ có duy nhất một cổng UART lộ ra ngoài qua đầu nạp, nên log dạng văn bản và dữ liệu ảnh nhị phân buộc phải đi chung một đường dây; nếu vẫn in log bằng Serial.printf như bản cũ thì middleware không thể phân biệt đâu là chữ đâu là byte ảnh và ảnh sẽ hỏng. Bài học này đã được kiểm chứng bằng chính sự cố ngày 13/09, khi mức log chi tiết làm nhiễu luồng nhị phân và đẩy chỉ số bytesSkipped lên cao.

Một điểm hay bị hiểu nhầm cần làm rõ trước khi vào các lưu đồ: Root không gửi từng mảnh lên máy tính mà ghép xong cả ảnh rồi mới gửi một khung IMAGE_FRAME duy nhất. Do đó middleware hoàn toàn không có bước ghép ảnh, và mọi con số "Frame ghép OK" hay "Frame bị huỷ" hiển thị trên dashboard đều lấy nguyên từ telemetry của Root. Hai "tầng số liệu" được ghi trong nhật ký ngày 17/09 thực chất là cùng một nguồn, chỉ khác thời điểm lấy mẫu. Middleware chỉ thống kê riêng ở mức khung UART các chỉ số framesOk, framesBadCrc và bytesSkipped — những chỉ số này đo chất lượng đường UART chứ không đo chất lượng sóng mesh.

5. Lưu đồ giải thuật

5.1. Giải thuật chụp và gửi ảnh trên node Leaf



Hình 4 — Lưu đồ hàm capture_and_send() trong node_camera.ino

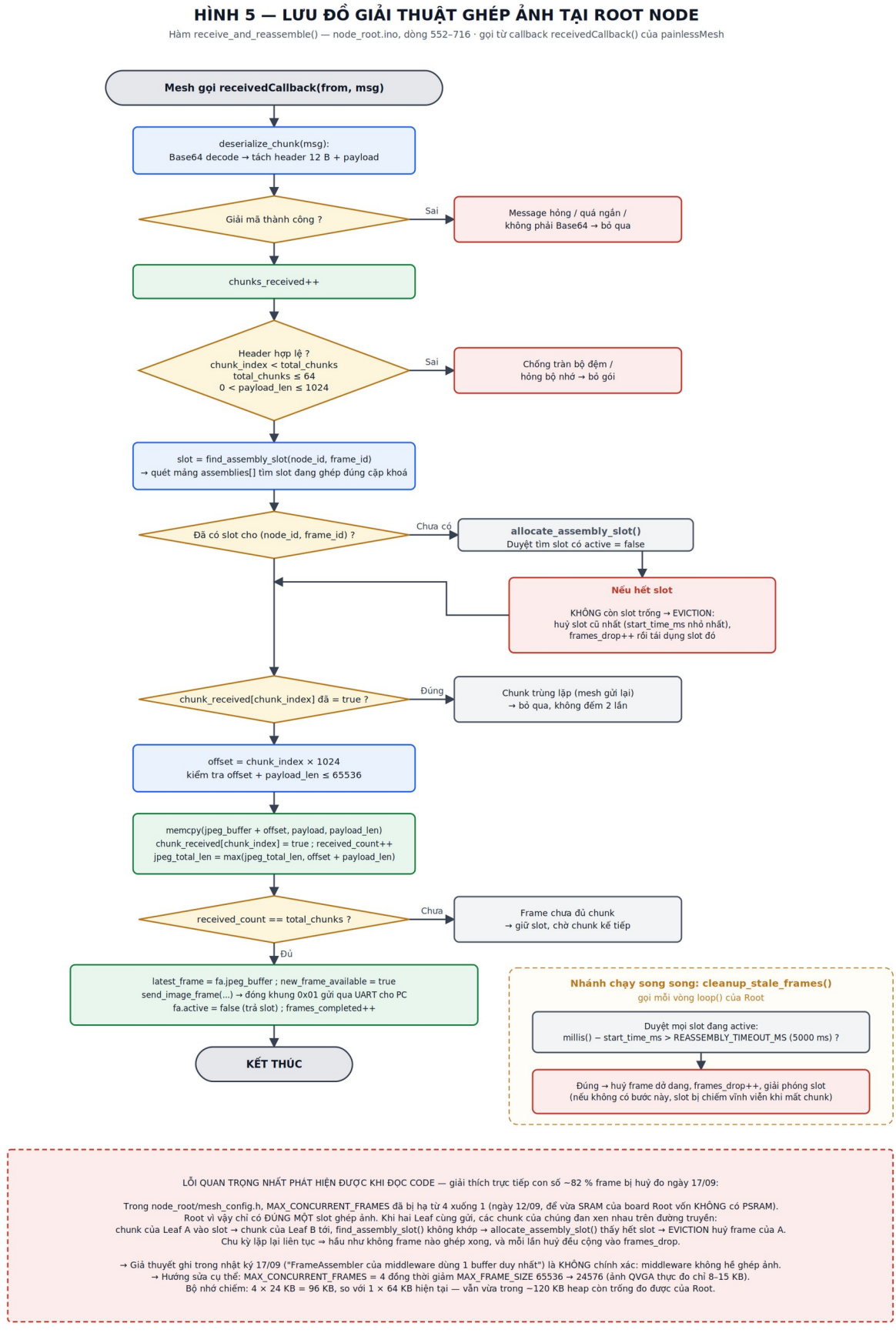
Hàm capture_and_send() nằm ở dòng 219 đến 405 của tệp node_camera.ino và được gọi từ vòng lặp chính mỗi 300 ms. Hàm bắt đầu bằng việc lấy một khung ảnh từ driver camera; lời gọi này chặn cho đến khi cảm biến xuất xong một khung và trả về con trỏ tới vùng nhớ ảnh trong PSRAM. Sau đó là ba phép kiểm tra nối tiếp: con trỏ có hợp lệ không, hai byte đầu có đúng là dấu hiệu bắt đầu ảnh JPEG 0xFF 0xD8 không, và kích thước ảnh có vượt ngưỡng cho phép không. Cả ba nhánh lỗi đều phải trả frame buffer về cho driver trước khi thoát.

Phần thân là một vòng lặp chạy từ mảnh số 0 đến mảnh cuối cùng. Với mỗi mảnh, chương trình tính vị trí bắt đầu đọc trong bộ đệm ảnh, tính độ dài thực tế của phần dữ liệu, đóng tiêu đề 12 byte, ghép tiêu đề với dữ liệu rồi mã hoá Base64 thành chuỗi, và cuối cùng phát chuỗi đó vào mạng mesh. Giữa hai lần gửi liên tiếp có một khoảng nghỉ 20 ms; khoảng nghỉ này là bắt buộc vì painlessMesh dùng TCP nội bộ giữa các node, gửi dồn dập sẽ làm tràn bộ đệm gửi và gói bị mất. Đáng chú ý là bên trong khoảng nghỉ vẫn gọi mesh.update() để mạng tiếp tục xử lý định tuyến và nhịp tim, tránh bị ngắt kết nối.

Từ đó rút ra công thức tốc độ khung hình, rất hữu ích khi hội đồng hỏi vì sao chỉ đạt 2–3 khung hình mỗi giây: tốc độ khung xấp xỉ 1000 chia cho tổng của chu kỳ chụp 300 ms và tích của số mảnh với 20 ms. Với một ảnh 10 KB chia thành 10 mảnh, kết quả là khoảng 2 khung hình mỗi giây — đúng bằng mức đã đo được trên dashboard.

Một điểm yếu phát hiện được khi đọc kỹ mã nguồn: hàm đang dùng mesh.sendBroadcast(), tức là phát gói tới toàn bộ node trong mạng, kể cả các Leaf khác vốn không có logic ghép ảnh nên chỉ nhận rồi bỏ đi. Khi hai hoặc ba Leaf cùng gửi, lượng gói phải truyền trên không khí bị nhân lên nhiều lần và mức tranh chấp kênh CSMA/CA nặng hơn hẳn mức cần thiết. Giải pháp là chuyển sang mesh.sendSingle(rootId, encoded) để gửi đích danh về Root. Ngay trong phần chú thích của mã nguồn cũng đã ghi nhận hướng tối ưu này nhưng chưa thực hiện.

5.2. Giải thuật ghép ảnh trên Root Node



Hình 5 — Lưu đồ hàm receive_and_reassemble() trong node_root.ino, kèm nhánh dọn dẹp chạy song song

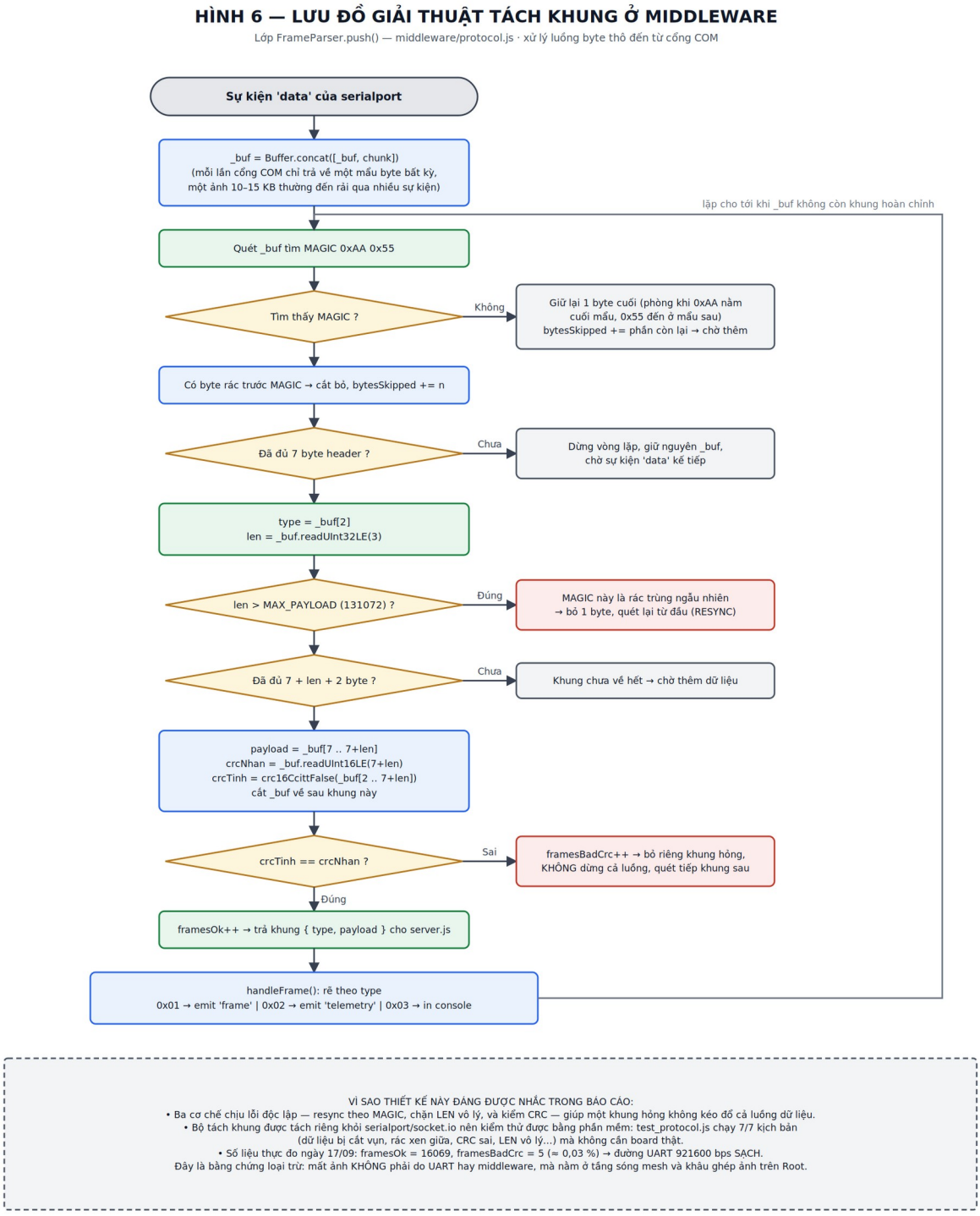
Đây là giải thuật trung tâm của đồ án, nằm ở dòng 552 đến 716 của node_root.ino và được gọi tự động mỗi khi painlessMesh nhận được một thông điệp. Mô hình lập trình ở đây là hướng sự kiện: hàm không được gọi theo lịch của người lập trình mà do hệ điều hành thời gian thực FreeRTOS lập lịch gọi lại — khác hẳn lối lập trình tuần tự quen thuộc trên vi điều khiển PIC.

Luồng xử lý gồm sáu bước. Bước một giải mã Base64 và tách ra tiêu đề cùng phần dữ liệu. Bước hai kiểm tra tính hợp lệ của tiêu đề, cụ thể là chỉ số mảnh phải nhỏ hơn tổng số mảnh, tổng số mảnh không vượt quá 64, và độ dài dữ liệu phải nằm trong khoảng hợp lệ — ba phép kiểm tra này phòng tràn bộ đệm và hỏng bộ nhớ. Bước ba tìm hoặc cấp phát một slot ghép ảnh theo cặp khoá gồm node_id và frame_id. Bước bốn bỏ qua các mảnh trùng lặp nhờ một bảng bit đánh dấu. Bước năm sao chép dữ liệu vào đúng vị trí trong bộ đệm ảnh. Bước sáu kiểm tra xem đã nhận đủ số mảnh chưa; nếu đủ thì đóng khung gửi qua UART cho máy tính và trả slot về trạng thái trống.

Song song với luồng trên, hàm cleanup_stale_frames() được gọi mỗi vòng lặp của Root để quét các slot đang ghép dở. Nếu một slot đã chờ quá 5 giây mà vẫn chưa đủ mảnh thì khung ảnh đó chắc chắn đã mất gói, slot bị huỷ và được giải phóng. Không có bước này thì chỉ cần mất một mảnh là slot sẽ bị chiếm vĩnh viễn.

Lỗi quan trọng nhất phát hiện được trong toàn bộ đợt kiểm tra mã nguồn nằm chính ở bước ba. Phân tích chi tiết được trình bày ở mục 6.2, và khung ghi chú màu đỏ trong Hình 5 giải thích trực tiếp con số 82 % khung hình bị huỷ đo được ngày 17/09.

5.3. Giải thuật tách khung ở middleware



Hình 6 — Lưu đồ lớp FrameParser.push() trong middleware/protocol.js

Cổng nối tiếp không trả về dữ liệu theo từng khung trọn vẹn mà trả về từng mẩu byte có độ dài bất kỳ, nên một ảnh 10–15 KB thường đến rải rác qua nhiều sự kiện. Lớp FrameParser giải quyết bài toán này bằng một bộ đệm tích luỹ và một vòng lặp tách khung có ba cơ chế chịu lỗi độc lập.

Cơ chế thứ nhất là đồng bộ lại theo dấu hiệu MAGIC. Mọi byte rác nằm trước cặp 0xAA 0x55 đều bị bỏ và cộng vào chỉ số bytesSkipped — tình huống này xảy ra khi middleware khởi động đúng lúc ESP32 đang gửi dở một khung. Cơ chế thứ hai là chặn giá trị độ dài vô lý: nếu trường LEN đọc được lớn hơn ngưỡng 131072 byte thì cặp MAGIC vừa tìm thấy chắc chắn là rác trùng ngẫu nhiên, chương trình bỏ đúng một byte rồi quét lại. Cơ chế thứ ba là kiểm tra CRC16 theo chuẩn CCITT-FALSE;

khung sai CRC bị bỏ riêng và đếm vào framesBadCrc, luồng dữ liệu vẫn tiếp tục chạy chứ không dừng.

Thuật toán CRC này được cài đặt hai lần ở hai ngôn ngữ khác nhau — bằng C trong tệp crc16.h của firmware và bằng JavaScript trong protocol.js của middleware — và bắt buộc phải cho ra cùng kết quả với cùng dữ liệu đầu vào. Việc tách bộ phân tích khung ra khỏi thư viện cổng nối tiếp và khỏi socket.io cho phép kiểm thử hoàn toàn bằng phần mềm: tệp test_protocol.js chạy bảy kịch bản gồm dữ liệu bị cắt vụn, rác xen giữa, CRC sai và LEN vô lý, tất cả đều đạt mà không cần board thật.

Kết quả đo thực tế ngày 17/09 xác nhận thiết kế này hoạt động đúng: framesOk đạt 16069 khung trong khi framesBadCrc chỉ có 5 khung, tương đương tỉ lệ lỗi khoảng 0,03 %. Đây là một bằng chứng loại trừ rất có giá trị: đường UART và middleware sạch, nên nguyên nhân mất ảnh chắc chắn nằm ở tầng sóng mesh hoặc ở khâu ghép ảnh trên Root.

6. Kết quả kiểm tra mã nguồn

6.1. Hiện trạng từng thành phần

Toàn bộ mã nguồn nằm trong thư mục D:\DOAN_CHUYENNGANH_MESH, đã được quản lý bằng Git với năm lần commit, lần gần nhất ngày 15/09 mang nội dung bring-up Leaf2 và Leaf3. Bảng dưới đây liệt kê tình trạng thực tế của từng tệp mã nguồn chính.

Tệp mã nguồn	Số dòng	Hiện trạng kiểm tra được
node_camera/node_camera.ino	571	Hoàn chỉnh, đã nạp và chạy ổn định trên cả ba board ESP32-CAM thật.
node_root/node_root.ino	1225	Hoàn chỉnh về chức năng; đã vá lỗi PSRAM và bổ sung khối gửi UART. Còn tồn tại hạn chế nghiêm trọng về số slot ghép ảnh (mục 6.2).
chunk_protocol.h / base64_util.h / crc16.h	314 / ~330 / 53	Ổn định, dùng chung cho cả hai firmware. Đã kiểm chứng gián tiếp qua mô phỏng Python và test JavaScript.
mesh_config.h (node_camera / node_root)	173 / 178	Hai bản khác nhau ở ba chỗ: SSID WiFi thật, MAX_CONCURRENT_FRAMES 4 so với 1, và phần chú thích lý do.
middleware/protocol.js	135	Hoàn chỉnh, thuần logic, kiểm thử được độc lập.
middleware/server.js	173	Hoàn chỉnh: mở cổng COM, tự kết nối lại sau 3 giây, phát Socket.IO, phục vụ dashboard, cung cấp API /status.
middleware/test_protocol.js	150	Bộ kiểm thử tự động, đạt 7/7 kịch bản.
middleware/public/dashboard.html	345	Chạy được, đã nối dữ liệu thật. Thiếu sơ đồ cây cha con và biểu đồ theo thời gian.
mo_phong_reassembly.py	206	Mô phỏng thuật toán ghép ảnh bằng Python, đạt 5/5 kịch bản gồm đúng thứ tự, sai thứ tự, trùng lặp, mất mảnh và tràn slot.
esp_idf_migration/.../ mesh_camera_main.c	206	Phase 1: đã biên dịch thành công (có tệp .bin trong thư mục build) nhưng ảnh vẫn là ảnh giả lập, chưa nối camera thật, chưa nạp lên board.
esp_idf_migration/.../ mesh_root_main.c	392	Phase 1 và Phase 4: đã có mã đo độ trễ qua trường capture_ts_ms và đo self-healing qua sự kiện bảng định tuyến, nhưng chưa chạy thật lần nào.

Về mặt tài liệu, dự án có ba tệp mô tả kỹ thuật là README.md, UART_PROTOCOL.md và README_MIGRATION.md, cùng năm nhật ký làm việc được ghi đều đặn từ 12/09 đến 17/09. Chất lượng tài liệu hoá này là một điểm mạnh rõ rệt: mỗi quyết định kỹ thuật đều có ghi lý do, mỗi giả thuyết sai đều được đính chính lại ở buổi sau thay vì để lại thông tin nhầm lẫn.

6.2. Lỗi trọng yếu phát hiện được: Root chỉ có một slot ghép ảnh

Đây là phát hiện quan trọng nhất của đợt kiểm tra mã nguồn lần này, và nó giải thích trực tiếp hiện tượng đã đo được ngày 17/09.

Trong tệp node_root/mesh_config.h, hằng số MAX_CONCURRENT_FRAMES đã bị hạ từ 4 xuống 1 vào ngày 12/09, với lý do hoàn toàn chính đáng ở thời điểm đó: board Root không có PSRAM, mỗi

slot chiếm MAX_FRAME_SIZE là 65536 byte, bốn slot sẽ cần 256 KB trong khi heap thực tế chỉ còn khoảng 120 KB. Hệ quả là Root hiện chỉ có đúng một slot ghép ảnh duy nhất.

Khi chỉ có một Leaf gửi, điều này không gây vấn đề gì: các mảnh của cùng một ảnh đến liên tiếp, ghép xong thì slot được trả lại ngay. Số liệu ngày 14/09 chứng minh điều đó với 1306 khung ghép thành công và chỉ 6 khung bị huỷ, tương đương 0,5 %. Nhưng khi hai Leaf cùng gửi, các mảnh của chúng đan xen nhau trên đường truyền. Một mảnh của Leaf A vào slot, rồi một mảnh của Leaf B đến; hàm find_assembly_slot() không tìm thấy slot khớp cặp khoá nên gọi allocate_assembly_slot(); hàm này thấy không còn slot trống nên thực hiện eviction — huỷ khung đang ghép dở của Leaf A, cộng một vào frames_drop, rồi tái dụng slot cho Leaf B. Ngay sau đó một mảnh khác của Leaf A lại đến và chu trình lặp lại theo chiều ngược lại.

Kết quả là hầu như không khung nào ghép xong được, đúng như số liệu đo: 111 khung thành công so với 496 khung bị huỷ, tức khoảng 82 %. Con số này nhất quán với cơ chế vừa mô tả chứ không phải ngẫu nhiên.

Điều này cũng đính chính một giả thuyết ghi sai trong nhật ký ngày 17/09. Nhật ký nêu nghi vấn "FrameAssembler ở middleware có khả năng đang dùng một buffer ghép dở duy nhất". Kiểm tra mã nguồn cho thấy middleware hoàn toàn không có lớp nào tên FrameAssembler và cũng không thực hiện ghép ảnh — Root gửi lên ảnh đã ghép xong. Nguyên nhân thật sự nằm ở firmware Root, không nằm ở middleware.

Hướng sửa cụ thể và có thể thực hiện ngay: nâng MAX_CONCURRENT_FRAMES từ 1 lên 4, đồng thời giảm MAX_FRAME_SIZE từ 65536 xuống 24576 byte. Mức 24 KB vẫn dư dả so với ảnh QVGA thực đo chỉ 8–15 KB. Tổng bộ nhớ chiếm khi đó là 4 nhân 24 KB bằng 96 KB, so với 64 KB của cấu hình hiện tại — mức tăng 32 KB này vẫn nằm trong khoảng 120 KB heap còn trống mà telemetry của Root báo về. Cần kiểm tra lại giá trị heap_free trong telemetry ngay sau khi sửa để xác nhận. Nếu heap không đủ, phương án thay thế là để 3 slot với MAX_FRAME_SIZE 24576, chiếm 72 KB.

Sau khi sửa, cần đo lại tỉ lệ khung bị huỷ với hai Leaf trước khi kết luận bất cứ điều gì về tranh chấp sóng. Bước này tách biến số phần mềm ra khỏi biến số vô tuyến, và chỉ khi tỉ lệ huỷ vẫn còn cao sau khi đã có đủ slot thì mới có cơ sở nói rằng nguyên nhân nằm ở tranh chấp CSMA/CA và cân nhắc các giải pháp nặng hơn như lệch pha thời điểm gửi hay chuyển luồng ảnh sang ESP-NOW.

7. Những phần đã hoàn thành tốt

Thứ nhất, toàn bộ chuỗi truyền dữ liệu từ camera đến trình duyệt đã thông suốt và đã được chứng minh bằng dữ liệu thật, không phải bằng mô phỏng. Ngày 14/09, với Leaf số 1 chạy một mình, hệ thống đạt 1306 khung ghép thành công trên 6 khung bị huỷ, tỉ lệ mất khung 0,5 %, chỉ số framesBadCrc bằng 0, và ảnh thật hiển thị liên tục trên dashboard. Việc đi trọn vẹn từ cảm biến ảnh qua mesh, qua UART, qua middleware đến trình duyệt là thành quả đáng kể với một người chưa từng thực hành mạng máy tính trước đó.

Thứ hai, chất lượng thiết kế giao thức ở mức cao hơn yêu cầu thông thường của một đồ án. Giao thức UART có đầy đủ cơ chế đồng bộ lại, chặn độ dài vô lý và kiểm tra CRC; giao thức chunk xử lý được cả ba tình huống khó là mảnh đến sai thứ tự, mảnh trùng lặp và mảnh bị mất. Quan trọng hơn, mỗi lựa chọn đều có lý do kỹ thuật viết sẵn — vì sao 921600 bps chứ không phải 115200, vì sao 1024 byte chứ không phải 1500, vì sao WebSocket chứ không phải MQTT, vì sao mạch tăng áp chứ không phải hạ áp. Đây chính là dạng câu hỏi hội đồng hay hỏi và hiện đã có sẵn câu trả lời.

Thứ ba, năng lực kiểm thử không cần phần cứng. Tệp mo_phong_reassembly.py mô phỏng đúng thuật toán ghép ảnh của Root bằng Python và đạt 5/5 kịch bản; tệp test_protocol.js kiểm thử bộ tách khung và đạt 7/7 kịch bản. Nhờ vậy, khi board thật gặp sự cố, có thể loại trừ ngay khả năng lỗi thuật toán và tập trung vào phần cứng hoặc vô tuyến. Đây là phương pháp làm việc đúng và nên được nêu rõ trong báo cáo cuối kỳ.

Thứ tư, quá trình chẩn đoán lỗi được thực hiện có phương pháp. Ba sự cố lớn đã được giải quyết đúng cách: lỗi treo do thiếu PSRAM trên board Root được xác định bằng cách đọc log thay vì đoán; lỗi mọi mảnh gửi thất bại được truy đến tận một lỗi lõi của thư viện painlessMesh và giải quyết bằng cách chuyển sang bản fork Alteriom; giả thuyết hai board trùng NodeID được bác bỏ dứt điểm bằng cách đọc trực tiếp ID trên từng board thay vì suy đoán. Mỗi lần như vậy đều để lại một kết luận rõ ràng trong nhật ký.

Thứ năm, tài liệu hoá đầy đủ và trung thực. Các hạn chế đều được ghi rõ thay vì che giấu — ví dụ ghi thẳng rằng danh sách node hiện là danh sách phẳng chứ không phải cây, và dashboard vẽ đúng những gì có thật thay vì vẽ một cây suy diễn. Thái độ này là điều hội đồng đánh giá cao.

8. Những thiếu sót cần bổ sung

Phần này xếp theo mức độ ưu tiên, dựa trên tiêu chí mức độ ảnh hưởng đến ba chỉ số hội đồng chấm điểm.

Ưu tiên	Nội dung còn thiếu	Vì sao quan trọng và hướng xử lý
1	Chưa hàn bộ nguồn pin 18650 độc lập cho ba Leaf	Đây là việc chặn đường lớn nhất. Còn cắm cáp USB thì các node bị buộc nằm gần nhau, không thể dựng chuỗi nhiều chặng, nên không đo được cả throughput theo số chặng lẫn self-healing kiểu đổi tuyến. Linh kiện đã đủ, quy trình 8 bước đã soạn, chỉ còn thực hiện.
2	Root chỉ có một slot ghép ảnh (MAX_CONCURRENT_FRAMES = 1)	Gây tỉ lệ huỷ khung 82 % khi có từ hai Leaf trở lên. Sửa bằng cách đặt 4 slot và giảm MAX_FRAME_SIZE xuống 24576 byte, sau đó đo lại ngay để xác nhận (phân tích đầy đủ ở mục 6.2).
3	Chưa có bất kỳ số liệu throughput nào	Đây là chỉ số trung tâm của đề tài. Cần bố trí vật lý theo chuỗi

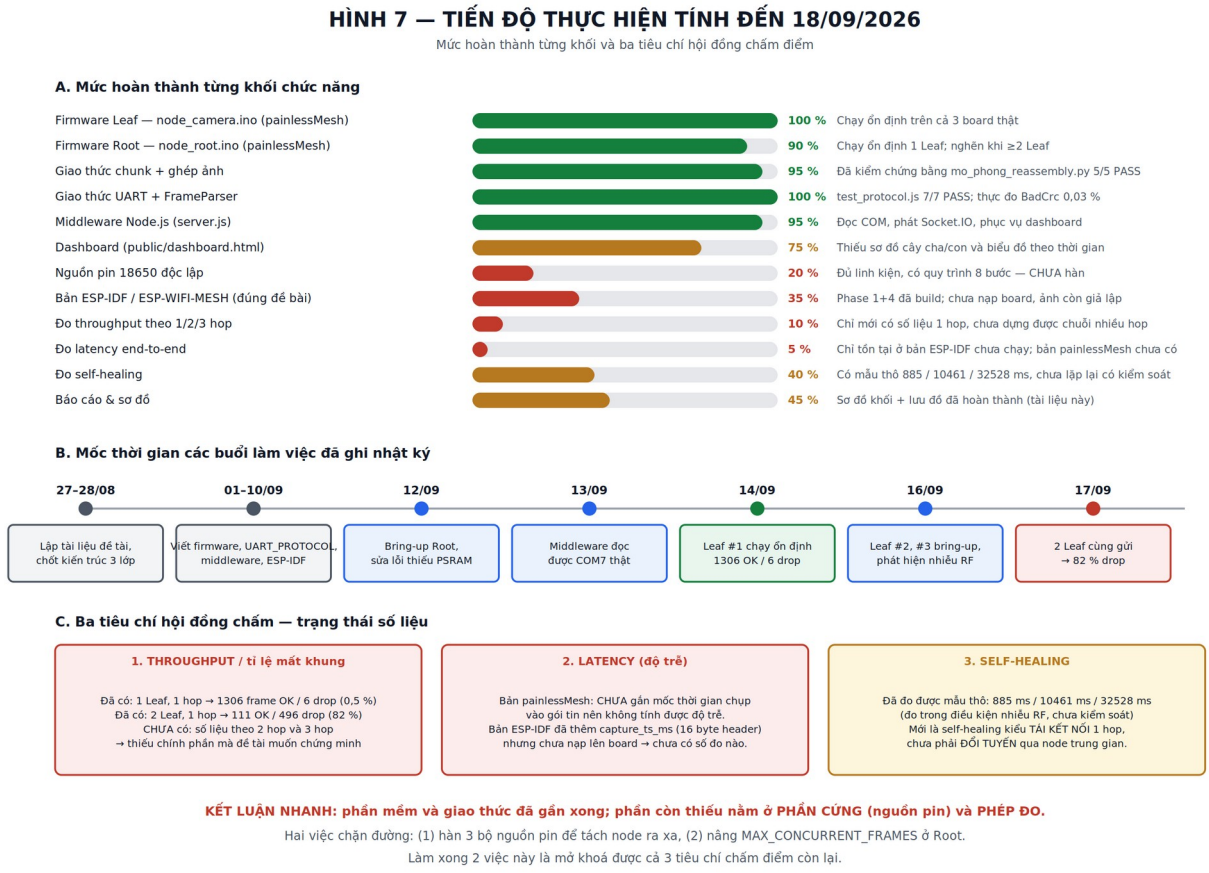
Ưu tiên	Nội dung còn thiếu	Vì sao quan trọng và hướng xử lý
	theo 2 hop và 3 hop	Root — node trung gian — node xa, sao cho node xa chỉ nghe được node trung gian. Phụ thuộc trực tiếp vào việc hoàn thành mục 1.
4	Chưa đo được độ trễ end-to-end	Bản painlessMesh hiện chưa gắn mốc thời gian chụp vào gói tin nên về nguyên tắc không tính được độ trễ. Cần bổ sung một trường capture_ts_ms vào ChunkHeader (mở rộng từ 12 lên 16 byte) giống như bản ESP-IDF đã làm, hoặc chuyển hẳn sang bản ESP-IDF.
5	Số liệu self-healing chưa có tính lặp lại	Mới có ba mẫu thô 885 ms, 10461 ms và 32528 ms đo trong điều kiện nhiễu RF không kiểm soát, độ phân tán quá lớn để đưa vào báo cáo. Cần lặp lại ít nhất 5 lần trong điều kiện ổn định, ghi min, max và trung bình.
6	Bản ESP-IDF / ESP-WIFI-MESH chưa từng nạp lên board	Đề bài ghi rõ yêu cầu cấu hình giao thức ESP-Mesh, trong khi painlessMesh là thư viện Arduino bên thứ ba chứ không phải API esp_mesh chính thức. Đây là rủi ro về mặt đáp ứng đề bài, cần quyết định sớm: hoặc hoàn tất chuyển đổi, hoặc chuẩn bị lập luận bảo vệ cho lựa chọn painlessMesh.
7	Leaf vẫn dùng sendBroadcast thay vì sendSingle	Làm tăng vô ích lượng gói trên không khí khi có nhiều Leaf. Sửa nhanh, chi phí thấp, nên làm cùng lúc với mục 2.
8	Dashboard chưa vẽ được cây topology thật	Cần Root báo cáo thêm quan hệ cha con, hiện đã có ghi chú TODO trong node_root.ino. Không nằm trong ba tiêu chí chấm nhưng là phần trực quan gây ấn tượng mạnh khi demo self-healing.
9	Mức log setDebugMsgTypes vẫn để ERROR | CONNECTION	Đang bật ở cả node_camera.ino (dòng 517) và node_root.ino (dòng 1100). Mức log chi tiết này làm nhiễu luồng UART nhị phân, đúng sự cố đã gặp ngày 13/09. Cần đổi lại thành ERROR trước khi bước vào giai đoạn đo số liệu.
10	Tính năng cầu nối WiFi của Root đang bị tắt	Dòng mesh.stationManual() đã bị chú thích lại ở dòng 1145 để tránh tranh chấp radio. Hệ quả là không xem được luồng MJPEG qua trình duyệt trên WiFi nhà. Không ảnh hưởng ba tiêu chí chấm, nhưng cần quyết định bỏ hẳn hay bật lại và nêu rõ trong báo cáo.
11	Hai tệp trạng thái tổng hợp đã lỗi thời	Tệp trang-thai-hien-tai.md dừng ở ngày 10/09 và migration-esp-wifi-mesh.md chưa đồng bộ với các phát hiện từ 12/09 đến 17/09. Cần cập nhật để tránh nhầm lẫn ở các buổi làm việc sau.

9. Tổng hợp số liệu đã đo được

Bảng dưới đây gom toàn bộ số liệu thật đã ghi nhận được tính đến ngày 18/09. Mọi ô trống đều được ghi rõ là chưa đo, không điền số ước lượng.

Chỉ số	Điều kiện đo	Giá trị	Nguồn và ghi chú
Khung ghép thành công / bị huỷ	1 Leaf, 1 hop	1306 / 6	Nhật ký 14/09. Tỉ lệ mất 0,5 % — mức bình thường.
Khung ghép thành công / bị huỷ	2 Leaf, 1 hop	111 / 496	Nhật ký 17/09. Tỉ lệ huỷ ~82 % — nguyên nhân ở mục 6.2.
Khung UART đúng / sai CRC	2 Leaf, 1 hop	16069 / 5	Nhật ký 17/09. Tỉ lệ lỗi ~0,03 % — đường UART sạch.
Thời gian tự phục hồi	Tái kết nối, 1 hop	885 / 10461 / 32528 ms	Nhật ký 16/09. Ba mẫu rời rạc trong điều kiện nhiễu RF, chưa đủ tin cậy.
Bộ nhớ heap còn trống trên Root	Sau cấp phát 1 slot	~122 600 byte	Telemetry 12/09. Ổn định, không thấy rò rỉ bộ nhớ.
Bộ nhớ heap trên Leaf	Khi chạy bình thường	~154 000 byte	Nhật ký 16/09. Tụt xuống 72 000 byte khi nguồn yếu.
Cường độ tín hiệu mesh	Leaf thấy Root	-57 dBm	Nhật ký 14/09. Đo ở khoảng cách vài chục centimét.
Tốc độ khung hình	1 Leaf	~2–3 khung/giây	Dashboard đo thực tế, khớp công thức tính ở mục 5.1.
Kích thước ảnh JPEG	QVGA, quality 12	8–15 KB	Đo từ log của firmware.
Throughput theo 2 hop và 3 hop	—	Chưa đo	Phụ thuộc nguồn pin độc lập.
Độ trễ end-to-end	—	Chưa đo	Bản painlessMesh chưa gắn mốc thời gian vào gói tin.

10. Đánh giá tiến độ và kế hoạch tiếp theo



Hình 7 — Tổng hợp tiến độ tính đến 18/09/2026

Nhìn tổng thể, phần mềm và giao thức của đồ án đã gần hoàn chỉnh: firmware chạy được trên phần cứng thật, giao thức đã kiểm thử, middleware và dashboard đã hoạt động với dữ liệu thật. Phần còn thiếu tập trung vào hai nhóm: phần cứng nguồn điện, và các phép đo. Điều đáng chú ý là hai nhóm này liên quan chặt với nhau — chính vì chưa có nguồn pin độc lập nên chưa tách được các node ra xa, và chính vì chưa tách được node nên chưa đo được hai trong ba chỉ số hội đồng chấm.

Đối chiếu với lộ trình 10 tuần trong đề cương gốc, công việc hiện đang nằm ở ranh giới giữa tuần 6–7 (middleware và dashboard) và tuần 8–9 (đo throughput, latency, self-healing). Phần tuần 8–9 mới chỉ bắt đầu và đang bị chặn bởi hai việc nêu trên. Về thời gian, nếu hoàn thành hai việc chặn đường trong vòng một tuần thì vẫn còn đủ dư địa cho giai đoạn đo và viết báo cáo.

Thứ tự công việc đề xuất cho tuần tới

Việc đầu tiên nên làm là sửa cấu hình slot ghép ảnh trên Root, vì chi phí thấp nhất mà tác động lớn nhất: chỉ cần đổi hai hằng số trong mesh_config.h, nạp lại firmware, chạy hai Leaf và đọc lại tỉ lệ huỷ khung. Nếu tỉ lệ này giảm mạnh thì đã xác nhận được nguyên nhân gốc và loại được biến số phần mềm ra khỏi bài toán. Nên làm cùng lúc việc đổi sendBroadcast thành sendSingle và đưa mức log về ERROR để một lần nạp firmware giải quyết được ba việc.

Việc thứ hai là hàn ba bộ nguồn pin theo đúng quy trình 8 bước đã soạn, làm thử một bộ trước và đo điện áp đầu ra bằng đồng hồ vạn năng trước khi cắm vào board, rồi mới làm hai bộ còn lại. Sau khi có nguồn độc lập, bố trí ba Leaf theo chuỗi Root — node trung gian — node xa, đặt sao cho node xa nằm ngoài tầm nghe trực tiếp của Root; đây là bố trí duy nhất cho phép đo throughput theo số chặng và quan sát self-healing kiểu đổi tuyến đúng nghĩa.

Việc thứ ba là chuẩn hoá quy trình đo. Với self-healing, cần đánh mốc thời gian lúc rút nguồn node trung gian, theo dõi node_count giảm rồi tăng lại, đọc giá trị last_self_heal_ms trong telemetry, và lặp lại ít nhất năm lần để lấy giá trị nhỏ nhất, lớn nhất và trung bình. Với throughput, đo lần lượt ở cấu hình 1 chặng, 2 chặng và 3 chặng trong cùng một điều kiện, mỗi cấu hình chạy đủ lâu để số liệu ổn định.

Việc cuối cùng cần quyết định sớm là số phận của bản ESP-IDF. Đề bài yêu cầu cấu hình giao thức ESP-Mesh, trong khi bản đang chạy dùng painlessMesh — một thư viện Arduino của bên thứ ba tự tổ chức cây qua TCP và JSON riêng, không phải API esp_mesh chính thức của Espressif. Bản ESP-IDF đã biên dịch thành công và đã có sẵn phần đo độ trễ lẫn self-healing, nhưng chưa từng nạp lên board và ảnh vẫn là ảnh giả lập. Có hai hướng: hoặc dành thời gian hoàn tất chuyển đổi để đáp ứng đúng chữ trong đề bài và đồng thời bỏ được lớp Base64 phình 33 %, hoặc giữ bản painlessMesh và chuẩn bị lập luận rằng painlessMesh vẫn là một giao thức mesh tự tổ chức đáp ứng đúng mục tiêu của đề tài. Nên hỏi ý kiến giảng viên hướng dẫn trước khi dồn thời gian vào một trong hai hướng.

11. Chuẩn bị trả lời hội đồng về phần đã làm

Dưới đây là những câu hỏi nhiều khả năng được đặt ra, gắn trực tiếp với các hình trong báo cáo này, kèm hướng trả lời ngắn gọn.

Câu hỏi có thể gặp	Hướng trả lời
Mạng này là mesh thật hay cây động?	Là cây động. painlessMesh tổ chức các node thành một cây, mỗi node chọn cha theo RSSI; không phải mọi node đều nối trực tiếp với mọi node như full mesh. Xem Hình 1.
Vì sao chọn chunk 1024 byte?	Nhỏ hơn MTU 1500 byte của 802.11 để chừa chỗ cho tiêu đề mesh và giảm rủi ro hỏng gói khi sóng yếu. Giảm nhỏ hơn nữa sẽ làm tăng số gói và tăng chi phí thời gian chiếm kênh. Xem Hình 3.
Mảnh ảnh mất giữa đường thì xử lý thế nào?	Slot ghép có bộ đếm thời gian 5 giây; quá hạn thì khung bị huỷ và slot được giải phóng. Mảnh đến sai thứ tự vẫn ghép đúng nhờ chunk_index. Mảnh trùng lặp bị bỏ nhờ bảng bit. Xem Hình 5.
Vì sao dùng baud 921600 chứ không phải 115200?	Ở 115200 bps, riêng UART chỉ tải được khoảng 11 KB mỗi giây, không đủ cho ảnh 10–15 KB ở 2–3 khung/giây, và UART sẽ thành nút thắt làm sai lệch chính phép đo throughput của mesh.
Vì sao dùng WebSocket thay vì MQTT?	Chỉ có một Root, một middleware, một dashboard nên không cần broker trung gian; WebSocket cho độ trễ thấp hơn và triển khai đơn giản hơn. MQTT là hướng mở rộng khi có nhiều Root.
Vì sao tốc độ chỉ 2–3 khung hình mỗi giây?	Do khoảng nghỉ 20 ms giữa các mảnh cộng với chu kỳ chụp 300 ms. Công thức và cách tính ở mục 5.1. Khoảng nghỉ này bắt buộc để không tràn bộ đệm TCP nội bộ của painlessMesh.
Anten PCB ảnh hưởng gì đến kết quả đo?	Làm hẹp cự ly mỗi chặng còn khoảng 2–5 mét thay vì 10–15 mét, nên không gian thử nghiệm phải thu nhỏ tương ứng. Đây là giới hạn đã ghi rõ của mô hình chứng minh nguyên lý.
Vì sao đổi từ OV2640 sang OV3660?	Cùng giao tiếp DVP 8 bit nên tương thích pinout; đổi vì

Câu hỏi có thể gặp	Hướng trả lời
	nguồn cung thực tế. Bù lại phải hạ độ phân giải xuống QVGA để giữ kích thước ảnh tương đương.

Cuối cùng, nếu hội đồng hỏi về con số 82 % khung bị huỷ, nên trả lời thẳng thắn theo hướng sau: đây không phải kết quả cuối cùng mà là một phát hiện trong quá trình đo, nguyên nhân đã được truy đến tận mã nguồn — Root chỉ có một slot ghép ảnh do phải thích ứng với board không có PSRAM — và hướng sửa đã xác định rõ với con số bộ nhớ cụ thể. Cách trình bày một lỗi đã hiểu rõ nguyên nhân bao giờ cũng thuyết phục hơn việc giấu nó đi.